

“Documentación sobre la solución de la cuarta actividad”



2026

Nombre del curso: SEMINARIO DE COMPLEMENTACIÓN PRÁCTICA I

Bloque del curso: PDSD-435

NRC: 4471

Estudiante: Johan Aamir Valentin Garcia

Instructor: Carlos Magno Mayna Aguilar

Índice

Contenido

Situación y solicitud.....	3
Preparar entorno (Herramientas)	3
Creación de migraciones	4
Desarrollo de modelos	5
Desarrollo de Controladores	6
Ajustando rutas	7
Creación de vistas	8
Vista de usuarios.....	8
Vista del CV	9
Pruebas del sistema	10

Situación y solicitud

El trabajo que realizaremos es una continuación de la situación que vimos en la anterior actividad, es decir, el de crear un sistema que permita registrar la asistencia de los estudiantes.

El sistema que desarrollaremos será con Laravel y base de datos mysql, así que podremos definirlos como una aplicación o sistema web.

Preparar entorno (Herramientas)

Para poder dar solución a la situación debemos preparar nuestro entorno de desarrollo. El IDE que usaremos para desarrollar el proyecto es visual studio code. Deberemos instalar los siguientes programas para poder continuar:

- **Node JS:** Permite actualizar de forma dinámica la vista del proyecto tanto css como js.
[Instalar Node JS](#)
- **XAMPP:** Es un paquete de software de entorno de desarrollo que puede funcionar en varios sistemas operativos, maneja programas como Apache, MySQL/MariaDB, PHP y Perl. Nos permitirá visualizar nuestra base de datos y trabajar con código PHP.
[Instalar XAMPP](#)
- **Composer:** Es un gestor de dependencias PHP para poder automatizar la instalación de librerías y paquetes.
[Instalar Composer](#)
- **Laravel:** Framework principal para el desarrollo del sistema, ofrece funciones de seguridad y permite la construcción de un sistema MVC por defecto.
[Instalar Laravel](#)

Cuando tengamos todos los programas instalados correctamente debemos asegurarnos de que trabajemos con versiones compatibles, en especial entre PHP y Composer. Ahora crearemos el proyecto, para eso debemos tener en cuenta la carpeta en donde lo desarrollaremos, una vez que creamos la carpeta en donde irá el proyecto, ejecutaremos cmd o powershell y ejecutaremos el siguiente comando:

```
laravel new [NombreProyecto]
```

Durante la creación del proyecto, es probable que te soliciten ciertas configuraciones, desde que tipo de marco de trabajo (framework) quieres usar, si deseas usar características de IA o que tipo de base de datos usaras.

Para este caso, usaremos phpunit como marco de trabajo, recomendaciones de IA y trabajaremos con base de datos mysql.

Dentro del archivo .env del proyecto, debemos configurar el nombre de la base de datos y crearla usando phpmyadmin brindada por XAMPP.

Creación de migraciones

Lo primero que se realiza en un proyecto Laravel tras su creación y configuración es realizar las migraciones. Las migraciones se pueden entender como consultas que se ejecutan en la base de datos para crear o realizar cambios, al contrario de las consultas normales en una base de datos, no se maneja scripts sql directa, sino que se usan sintaxis php y se quedan almacenados para poder ejecutarlos cuando se llamen.

Laravel ofrece dos migraciones por defecto, el cual es de usuario, así que usaremos esta migración para trabajar.

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
return new class extends Migration
{
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->rememberToken();
            $table->timestamps();
        });

        Schema::create('password_reset_tokens', function (Blueprint $table) {
            $table->string('email')->primary();
            $table->string('token');
            $table->timestamp('created_at')->nullable();
        });

        Schema::create('sessions', function (Blueprint $table) {
            $table->string('id')->primary();
            $table->foreignId('user_id')->nullable()->index();
            $table->string('ip_address', 45)->nullable();
            $table->text('user_agent')->nullable();
            $table->longText('payload');
            $table->integer('last_activity')->index();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('users');
        Schema::dropIfExists('password_reset_tokens');
        Schema::dropIfExists('sessions');
    }
};
```

- **Función up:** En esta parte de la migración se realizan todos los métodos de creación principalmente.
- **Schema::create:** Dentro de este método se define el nombre de la tabla y los atributos que este tendrá. La variable \$table permite definir el nombre del atributo, que tipo de dato es y las restricciones que pueden tener.
- **Función down:** Es la función que se ejecuta de forma auxiliar, trabajando principalmente para limpiar tablas creadas anteriormente.
- **Schema::dropIfExists:** Se encarga de eliminar ciertas tablas si es que existen, esto permite evitar errores al crear tablas con las tablas anteriores.

Desarrollo de modelos

Los modelos son una parte fundamental del patrón MVC, aquí es donde se definen los atributos dentro del sistema y sobre todo las relaciones que hay entre modelos.

Como estamos usando la migración de usuario por defecto de Laravel, vamos a también usar el modelo por defecto que nos brinda.

User.php:

```
<?php
namespace App\Models;

use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    use HasFactory, Notifiable;

    protected $fillable = [
        'name',
        'email',
        'password',
    ];

    protected $hidden = [
        'password',
        'remember_token',
    ];

    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
        ];
    }
}
```

- **Use HasFactory:** Llama al archivo factory de user. Un factory en Laravel permite que Tinker pueda meter registros de forma automatizada teniendo en cuenta los atributos, los tipos de datos y sus relaciones.
- **Protected \$fillable:** Es un arreglo o lista que le indica a Laravel que campos puede rellenar automáticamente a través de un formulario (vista). Dentro de este se deben mencionar los campos o atributos que se tomarán en cuenta para rellenarse, en este caso, serán el nombre, el email y la contraseña del usuario.
- **Protected \$hidden:** Asegura que los datos de la contraseña y el token para ocultarlos y mantener la seguridad de la información. El token es una llave única por usuario que permite que los usuarios registrados puedan identificarse al iniciar sesión.
- **Función casts:** Permite hacer acciones automáticamente tras realizar un registro, la contraseña se hashea para que no pueda ser vista puramente y el

email_verified_at genera una fecha automáticamente tomando el tiempo real en el que el usuario registrado ingreso al sistema.

Desarrollo de Controladores

El controlador es la segunda parte del patrón MVC, su principal función en el desarrollo del sistema es definir el comportamiento que tendrá el sistema, es decir, que acciones harán al actuar con el usuario, aquí se manejan las partes del CRUD.

Para este proyecto, solamente trabajaremos con los método index, store y destroy.

```
<?php
namespace App\Http\Controllers;
use Illuminate\Support\Facades\Hash;
use App\Models\User;
use Illuminate\Http\Request;

class UserController extends Controller
{
    public function index()
    {
        $users = User::latest()->paginate(5);
        return view('users.index', ['users'=>$users]);
    }

    public function store(Request $request)
    {
        $request->validate([
            'name'=>'required|max:30',
            'email'=>'required|email|max:255|unique:users',
            'password'=>'required|max:255',
        ]);
        User::create($request->only('name','email','password'));
        return back();
    }

    public function destroy(User $user)
    {
        $user->delete();
        return back();
    }
}
```

- **Función index:** Es la función que mostrará los registros de los usuarios presentes en la base de datos. La variable \$users llama a los últimos 5 registros para mostrarlos. De ahí regresa la vista, aquí es donde se define el nombre de la vista o llamado en Laravel como Blade que se mostrará para que ejecute este método. Usa users.index, es decir, buscará en la carpeta views/users/index.blade.php.
- **Función store:** Aquí trabaja con request, es decir, recibe una petición del usuario (método POST). La variable \$request el cual traerá los datos del formulario, va a validarlos por el tipo de dato, nombre de campo y sus restricciones como su tamaño. Al terminar la validación, va a ejecutar un método create dentro del modelo User y tomará los datos recolectados del request para guardarlo en la base de datos. Finalizando, regresará al index.
- **Función destroy:** Esta función solo traerá el id del usuario al que se desea hacer los cambios, así que por ello se llama al modelo User. La variable \$user (de parte del modelo) va a ejecutar el método delete, eliminando el registro y finalmente volviendo al index nuevamente.

Ajustando rutas

Las rutas en un proyecto Laravel ayudan a poder unir y comunicar el controlador con las vistas, a partir de esta comunicación es por la que los datos ingresados en un formulario pueden almacenarse en la base de datos.

Entraremos a la carpeta routes y dentro de esta abriremos el archivo web.php, aquí se ubican nuestras rutas. Por defecto laravel llama a la vista llamada 'welcome' como página de inicio, lo que nosotros haremos es cambiar esas rutas por las siguientes.

Web.php:

```
<?php

use App\Http\Controllers\UserController;
use Illuminate\Support\Facades\Route;

Route::get('/', [UserController::class, 'index']);
Route::resource('users',UserController::class);
Route::view('/cv', 'cv')->name('cv');
```

- **Route::get:** Permite ejecutar el método store del controlador user.
- **Route::resource:** Llama a la vista index del controlador user para mostrarlo como página de inicio.
- **Route::view:** Establece el acceso a la vista cv para poder acceder a ella a través del url.

Todos los nombres de las vistas deben seguir las que aparecen en el controlador, en este caso, las vistas que deberemos crear son cv.blade.php y index.blade.php.

Creación de vistas

Vista de usuarios

En nuestro proyecto Laravel entraremos al siguiente directorio resources/views. Dentro vamos a crear la carpeta users, la misma que mencionamos en nuestra ruta y controlador. Aquí tendremos nuestro archivo index.blade.php en donde tendremos nuestra tabla para mostrar los registros, el botón para eliminar un registro específico y el formulario para crear un nuevo usuario.

```
<table class="text-center min-w-full">
  <thead class="bg-blue-500 text-white">
    <tr>
      <th class="py-2 px-2">ID</th>
      <th class="py-2 px-2">NOMBRE</th>
      <th class="py-2 px-2">EMAIL</th>
      <th class="py-2 px-2">ACCIONES</th>
    </tr>
  </thead>
  <tbody>
    <@foreach ($users as $user)>
      <tr>
        <td class="py-2 px-2">{{ $user->id }}</td>
        <td class="py-2 px-2">{{ $user->name }}</td>
        <td class="py-2 px-2">{{ $user->email }}</td>
        <td class="py-2 px-2">
          <form action="{{ route('users.destroy', $user) }}" method="POST"
            style="display:inline">
            @csrf
            @method('DELETE')
            <button class="px-3 py-1 text-sm" type="submit" onclick="return
              confirm('¿Desea eliminar este usuario?')">
              Eliminar
            </button>
          </form>
        </td>
      </tr>
    </foreach>
  </tbody>
</table>
<div class="mt-4">
  {{ $users->links() }}
</div>
```

De entre toda la tabla, lo que le da la lógica es el bucle foreach, este bucle va a recopilar todos los registros que estén dentro de la base de datos para mostrarlo. A través de {{ }} se puede llamar a un dato específico de la tabla usuario, los cuales son el id, el nombre y el correo.

Aquí también colocaremos un botón, este botón tendrá una acción, el cual llamará al método destroy tomando de referencia el id del registro. Una división afuera de la tabla va a encargarse del paginado de registros.

Con esto ya llevamos la lógica del index, ahora vamos a crear un formulario para poder crear nuevos registros.

```
<form action="{{ route('users.store') }}" method="POST">
  @csrf
  <p class="font-bold text-2xl">Añadir Usuario</p>
  <div class="pb-4">
    <p class="font-bold text-md">Nombre:</p>
    <input type="text" name="name" class="border border-gray-500 rounded-lg"
      placeholder="Ingrese nombre">
    </div>
  <div class="pb-4">
    <p class="font-bold text-md">Email:</p>
    <input type="text" name="email" class="border border-gray-500 rounded-lg"
      placeholder="Ingrese nombre">
    </div>
  <div class="pb-4">
    <p class="font-bold text-md">Contraseña:</p>
    <input type="password" name="password" class="border border-gray-500 rounded-
      lg" placeholder="Ingrese nombre">
    </div>
  <button class="py-2 px-4 rounded bg-blue-400 hover:bg-blue-500 transition-all"
    type="submit">
    Guardar
  </button>
  <button class="py-2 px-4 rounded bg-gray-300 hover:bg-gray-400 transition-all"
    type="reset">
    Vaciar
  </button>
</form>
```


En este formulario debemos realizar los siguientes ajustes, el primero es indicar la acción que realizará, en este caso, va a ejecutar el método store. Para poder llevar los datos al controlador usaremos POST. De ahí en fuera, no debemos ajustar nada más que los inputs que deben seguir el tipo de dato con los que están definidos los campos y que tengan como name los nombres definidos en el controlador. El botón submit ya hará el trabajo de enviar los cambios.

Vista del CV

Cómo siguiente actividad vamos a realizar un cv con html y css, para lograrlo, usaremos tailwind (css de Laravel) y node js para así visualizar los cambios a tiempo real. Dentro del mismo directorio de resources/views vamos a crear el archivo cv.blade.php.

Aquí dividiremos la estructura de la página en tres, una 'cabecera' donde mostrará el nombre de la persona y su imagen. De ahí abran dos columnas, la primera contendrá información de las habilidades de la persona, mientras que la otra columna va a mostrar los proyectos realizados por la persona y los estudios que tuvo. El orden y diseño dependerán completamente de los gustos.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  @vite(['resources/css/app.css', 'resources/js/app.js'])
  <title>Curriculum</title>
</head>
<body>
  <div id="header" class="max-w-6xl mx-auto">
    <div class="border border-gray-400 bg-blue-300 rounded-lg p-6">
      
      <p class="text-4xl font-bold">Johan Amir Valentin Garcia</p>
    </div>
  </div>

  <div id="body" class="grid grid-cols-3 gap-10 max-w-6xl mx-auto py-6">
    <div class="col-span-1 border border-gray-400 bg-blue-100 rounded-lg p-6">
      <div class="pb-4">
        <p class="text-lg font-bold">PERFIL PROFESIONAL</p>
        <p class="text-md">
          Soy estudiante de la carrera de Desarrollo de Software, con conocimientos básicos en desarrollo web frontend y backend, base de datos relacionales. Busco oportunidades para aplicar mis habilidades en entornos profesionales y crear soluciones creativas.
        </p>
      </div>
      <div class="pb-4">
        <p class="text-lg font-bold">CONTACTOS</p>
        <ul class="text-md">
          <li>+51 943 896 447</li>
          <li>amirvalentin718@gmail.com</li>
          <li>Elemento 3</li>
        </ul>
      </div>
      <div class="pb-4">
        <p class="text-lg font-bold">IDIOMAS</p>
        <ul class="text-md">
          <li>Español: Nativo</li>
          <li>Inglés: Básico (A2)</li>
        </ul>
      </div>
      <div class="pb-4">
        <p class="text-lg font-bold">PROGRAMAS</p>
        <ul class="text-md">
          <li>Lenguajes de programación y etiquetas (básico): Java, HTML, PHP</li>
          <li>Entornos de Desarrollo Integrado (básico): Eclipse, VS Code, Sublime Text</li>
          <li>Base de datos (básico): MySQL, SQL Server, Oracle SQL</li>
        </ul>
      </div>
      <div class="pb-4">
        <p class="text-lg font-bold">HABILIDADES</p>
        <ul class="text-md">
          <li>Modelado y diseño de base de datos relacionales</li>
          <li>Pensamiento creativo</li>
          <li>Programación Web con patrón MVC</li>
        </ul>
      </div>
    </div>

    <div class="col-span-2 border border-gray-400 bg-white rounded-lg p-6">
      <div class="pb-4">
        <p class="text-lg font-bold">FORMACIÓN ACADÉMICA</p>
        <p>Carrera profesional de desarrollo de Software - SENATI</p>
      </div>
      <div class="pb-4">
        <p class="text-lg font-bold">PROYECTOS DE DESARROLLO</p>
        <p class="text-md font-bold">Página Web simple para una editorial</p>
        <p class="text-md">Implementación del Framework Bootstrap 4, HTML y CSS para el frontend la página Web.</p>
        <p class="text-md font-bold">Sistema Web de gestión de empleados</p>
        <p class="text-md">Creación de un sistema Web para agregar, modificar y eliminar registro de empleados. Con backend Java, frontend HTML y base de datos MySQL con patrón MVC.</p>
        <p class="text-md font-bold">Página Web para la lectura de EPUB</p>
        <p class="text-md">Desarrollo de una página Web con la implementación de la librería epub para la lectura de libros electrónicos en la Web.</p>
      </div>
      <div class="pb-4">
        <p class="text-lg font-bold">CURSOS Y BADGES</p>
        <p class="text-md font-bold">CISCO</p>
        <p class="text-md">Introducción al Internet de las Cosas, introducción a las Tecnologías de la Información, conceptos básicos de Hardware, introducción a la ciencia de datos, introducción a la ciberseguridad, conceptos básicos de redes, Python Essentials, C Essentials.</p>
        <p class="text-md font-bold">MySQL/SPARK</p>
      </div>
    </div>
  </div>
</body>
</html>
```

Pruebas del sistema

Con todo el desarrollo del proyecto nos quedaríamos con el siguiente resultado.

--Usuarios--

[Ver curriculum](#)

Añadir Usuario

Nombre:

Email:

Contraseña:

ID	NOMBRE	EMAIL	ACCIONES
52	pancho sanchez	frankierivers@example.com	Eliminar
51	Juan Perez	juampe@example.org	Eliminar
1	Jaren Barton DDS	idella.zboncak@example.org	Eliminar
2	Ethel Miller V	germaine79@example.com	Eliminar
3	Mrs. Bette Haley Sr.	chebie39@example.net	Eliminar

Showing 1 to 5 of 50 results

< 1 2 3 4 5 6 7 8 9 10 >

Ilustración 1 Página de inicio del sistema



Ilustración 2 URL de la página de inicio o index


Johan Aamir Valentin Garcia

PERFIL PROFESIONAL
Soy estudiante de la carrera de Desarrollo de Software, con conocimientos básicos en desarrollo web frontend y backend, base de datos relacionales. Busco oportunidades para aplicar mis habilidades en entornos profesionales y crear soluciones creativas.

CONTACTOS
• +51 943 896 447
• aamirvalentin718@gmail.com
• Elemento 3

IDIOMAS
• Español: Nativo
• Inglés: Básico (A2)

PROGRAMAS
• Lenguajes de programación y etiquetas (básico): Java, HTML, PHP.

FORMACIÓN ACADÉMICA
Carrera profesional de desarrollo de Software – SENATI

PROYECTOS DE DESARROLLO

Página Web simple para una editorial
• Implementación del Framework Bootstrap 4, HTML y CSS para el frontend de la página Web.

Sistema Web de gestión de empleados
• Creación de un sistema Web para agregar, modificar y eliminar registros de empleados. Con backend Java, frontend HTML y base de datos MySQL con patrón MVC.

Página Web para la lectura de EPUB
• Desarrollo de una página Web con la implementación de la librería epubjs para la lectura de libros electrónicos en la Web.

CURSOS Y BADGES
CISCO:
Introducción al Internet de las Cosas, introducción a las Tecnologías de la Información, conceptos básicos de Hardware, introducción a la ciencia de datos, introducción a la ciberseguridad, conceptos básicos de redes, Python Essentials, C Essentials.
MyEIT/SPARK.

Ilustración 3 Página del cv



Ilustración 4 URL de la página del cv

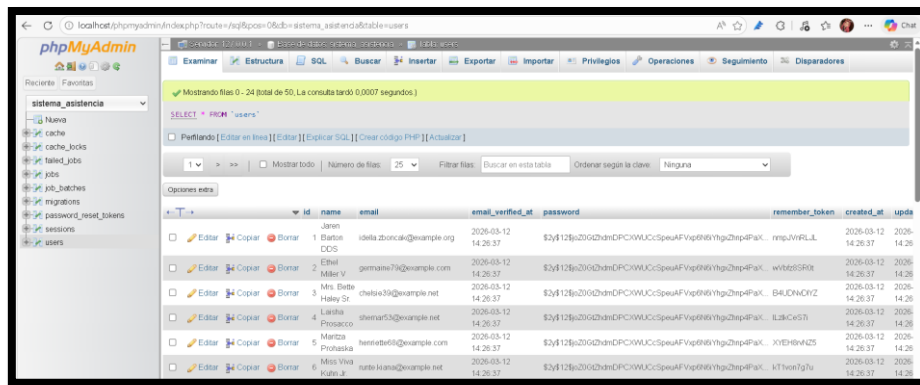


Ilustración 5 Página phpmyadmin para ver los registros creados

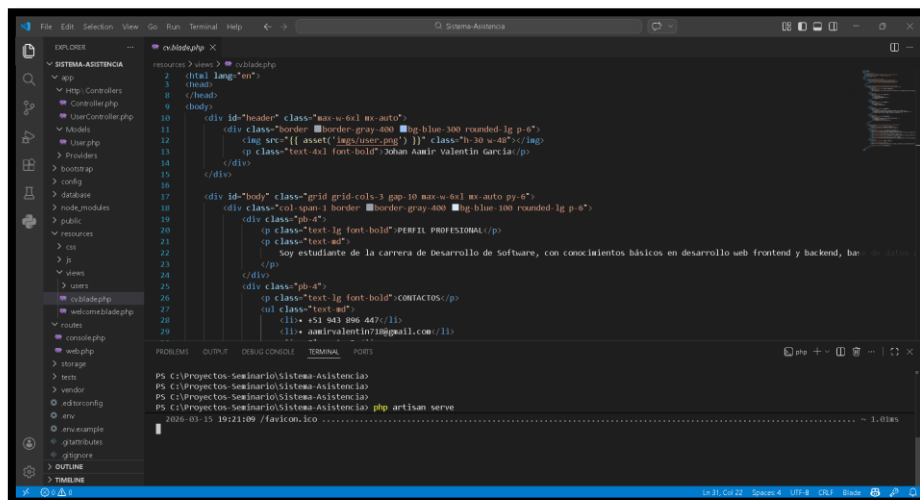


Ilustración 6 Evidencia del desarrollo del sistema